

Module 21 Structured Notes: Circular Doubly Linked Lists

1. Current Progress Analysis

You are now at **Module 21: Circular Doubly Linked Lists**.

Before this module, you already learned these important linked-list ideas:

Previous topic	Why it matters in Module 21
Singly linked lists	You know that a node can point to the next node.
Circular singly linked lists	You know that the last node can point back to <code>Head</code> instead of <code>NULL</code> .
Doubly linked lists	You know that each node has <code>prev</code> , <code>data</code> , and <code>next</code> .
Doubly insertion	You know insertion must update both forward and backward links.
Doubly deletion	You know deletion must preserve both <code>prev</code> and <code>next</code> links.
Doubly reversal	You know both links can be changed carefully without losing the list.

Module 21 combines two ideas that you already know separately:

1. **Circular structure** from circular linked lists.
2. **Two-way movement** from doubly linked lists.

So this module is about a list that can move:

- forward using `next`,
- backward using `prev`,
- and around in a circle without reaching `NULL`.

2. Big Idea of Module 21

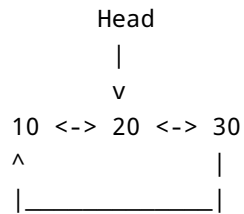
A **circular doubly linked list** is a linked list where:

1. Every node has a `prev` pointer.
2. Every node has a `next` pointer.
3. The last node points forward to the first node.
4. The first node points backward to the last node.

A normal linear doubly linked list looks like this:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

A circular doubly linked list looks like this:



The two most important rules are:

```
last->next == Head;
Head->prev == last;
```

That means:

- From the last node, moving forward returns to `Head`.
- From `Head`, moving backward reaches the last node.
- There is no `NULL` at the front or back unless the list is empty.

3. Learning Goals

By the end of Module 21, you should be able to:

1. Define a circular doubly linked list.
2. Draw an empty, one-node, and multi-node circular doubly linked list.
3. Explain the role of `Head`.
4. Explain why `Head->prev` points to the last node.
5. Display a circular doubly linked list using `do-while`.
6. Insert a node before `Head` in $O(1)$ time.
7. Insert a node after a given index.
8. Handle empty-list and one-node-list edge cases.
9. Explain why general insertion needs four link updates.
10. Analyze time and space complexity.

4. Node Structure

A circular doubly linked list uses the same node structure as a normal doubly linked list.

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

Each node has three parts:

Field	Meaning
prev	Stores the address of the previous node.
data	Stores the actual value.
next	Stores the address of the next node.

Simple node picture:

[prev | data | next]

Example node storing 20:

[address of 10 | 20 | address of 30]

This means:

20->prev points to 10
20->next points to 30

5. Why prev and next Must Be Pointers

This is correct:

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

This is wrong:

```
struct Node {
    struct Node prev;    // wrong
    int data;
    struct Node next;    // wrong
};
```

Why is it wrong?

Because that would mean:

```
A Node contains a full Node.
That full Node contains another full Node.
That full Node contains another full Node.
And so on forever.
```

The compiler cannot create a structure with infinite size.

So we use pointers:

```
struct Node *prev;
struct Node *next;
```

A pointer stores only an address, so the node has a fixed size.

6. Meaning of Head

In circular linked lists, the starting pointer is usually called **Head**.

```
struct Node *Head = NULL;
```

Head points to the first node that we choose as the starting point.

Example:

```
Head
|
v
10 <-> 20 <-> 30
```



Important point:

A circular list does not naturally end at `NULL`.

So `Head` is used for two jobs:

1. It tells us where to start.
2. It tells us where to stop after one full round.

The basic circular traversal rule is:

Start at Head.
Move node by node.
Stop when you return to Head.

7. Linear Doubly List vs Circular Doubly List

Feature	Linear doubly linked list	Circular doubly linked list
First node's <code>prev</code>	<code>NULL</code>	Last node
Last node's <code>next</code>	<code>NULL</code>	<code>Head</code>
Natural ending	Yes, at <code>NULL</code>	No natural <code>NULL</code> ending
Forward traversal	Stops at <code>NULL</code>	Stops when back at <code>Head</code>
Backward traversal from first node	Not possible directly	Goes to last node using <code>Head->prev</code>
Shape	Straight line	Circle

Linear doubly linked list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Circular doubly linked list:

```
10 <-> 20 <-> 30
^         |
|         |
```

8. Important Invariants

An **invariant** is a rule that must stay true while the list is correct.

For a circular doubly linked list, the main invariants are:

Rule 1: The last node points forward to `Head`

```
last->next == Head;
```

Rule 2: `Head` points backward to the last node

```
Head->prev == last;
```

Rule 3: Neighbor links must agree

If node `A` comes before node `B`, then:

```
A->next == B;
B->prev == A;
```

Picture:

```
A <-> B
```

Rule 4: A one-node list points to itself

If there is only one node:

```
Head->next == Head;
Head->prev == Head;
```

These rules help you check whether your list is still correctly connected.

9. Empty, One-Node, and Multi-Node Cases

A circular doubly linked list has three important structural cases.

Case 1: Empty Circular Doubly Linked List

An empty list has no nodes.

```
Head == NULL
```

Picture:

```
Head -> NULL
```

There is no first node. There is no last node. There is nothing to display or insert after.

Case 2: One-Node Circular Doubly Linked List

Suppose the only value is 10.

Picture:

```
Head
|
v
10
^|
||
|_|
```

For one node:

```
Head->next = Head;
Head->prev = Head;
```

This means:

- The node after Head is Head itself.
- The node before Head is also Head itself.
- The node is both the first node and the last node.

This is different from a normal linear doubly linked list.

Linear one-node doubly list:

```
first->prev = NULL;  
first->next = NULL;
```

Circular one-node doubly list:

```
Head->prev = Head;  
Head->next = Head;
```

Case 3: Multi-Node Circular Doubly Linked List

Example:

```
Head  
|  
v  
10 <-> 20 <-> 30  
^           |  
|_____||
```

The links are:

```
10->prev == 30;  
10->next == 20;  
  
20->prev == 10;  
20->next == 30;  
  
30->prev == 20;  
30->next == 10;
```

So from `Head`, you can move forward:

```
p = p->next;
```

And you can move backward:


```
p = p->prev;
```

10. Forward and Backward Movement

Because each node has both `prev` and `next`, a circular doubly linked list can move in two directions.

Forward movement

```
p = p->next;
```

Example:

```
10 -> 20 -> 30 -> 10 -> 20 -> ...
```

Backward movement

```
p = p->prev;
```

Example:

```
10 -> 30 -> 20 -> 10 -> 30 -> ...
```

Important:

Since the list is circular, movement can continue forever if you do not use a stopping condition.

11. Why Normal Display Does Not Work

For a normal linear linked list, display often uses this condition:

```
while (p != NULL)
```

That works for a linear list because the last node points to `NULL`.

Example:

```
10 -> 20 -> 30 -> NULL
```

But a circular doubly linked list does not normally reach `NULL`.

Example:

```
10 <-> 20 <-> 30
^         |
|         |
|         |
```

If you write this:

```
while (p != NULL) {
    printf("%d ", p->data);
    p = p->next;
}
```

The loop may run forever because `p` keeps moving around the circle.

Correct idea:

```
Stop when p returns to Head.
```

12. Why `do-while` Is Needed

For circular display, the stopping condition is:

```
p != Head
```

But there is a beginner trap.

At the start:

```
p = Head;
```

So immediately:

```
p == Head
```

If you write this:

```
while (p != Head) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

The loop will not run even once because `p` is already equal to `Head` at the beginning.

So we use `do-while`.

A `do-while` loop:

1. Runs the body first.
2. Checks the condition after.

That is exactly what circular display needs.

Simple rule:

```
Print first.  
Move next.  
Then check whether you returned to Head.
```

13. Display Function

```
void Display(struct Node *h) {  
    if (h == NULL) {  
        return;  
    }  
  
    struct Node *p = h;  
  
    do {  
        printf("%d ", p->data);  
        p = p->next;  
    } while (p != h);  
}
```

```
    printf("\n");  
}
```

Explanation

```
if (h == NULL) {  
    return;  
}
```

If the list is empty, there is nothing to print.

```
struct Node *p = h;
```

Use a temporary pointer `p` to move through the list. Do not move `Head` directly.

```
do {  
    printf("%d ", p->data);  
    p = p->next;  
} while (p != h);
```

This prints each node once and stops when `p` returns to the starting node.

14. Dry Run: Display

Given list:

```
Head  
|  
v  
6 <-> 9 <-> 2  
^           |  
|_____|
```

Function call:

```
Display(Head);
```

Start:

p points to 6

Trace:

Step	p points to	Action
1	6	Print 6, move to 9.
2	9	Print 9, move to 2.
3	2	Print 2, move to 6.
Stop	6	p == Head, so stop.

Output:

6 9 2

15. Length Function

To insert by index, we need to know how many nodes are in the list.

In a linear list, length often uses:

```
while (p != NULL)
```

But that is wrong for a circular list.

A circular list should stop when it returns to Head.

```
int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);
}
```

```
    return len;
}
```

Explanation

```
if (h == NULL) {
    return 0;
}
```

An empty list has length `0`.

```
int len = 0;
struct Node *p = h;
```

`len` counts nodes. `p` moves through the list.

```
do {
    len++;
    p = p->next;
} while (p != h);
```

Count the current node. Move forward. Stop when we return to the starting node.

16. Dry Run: Length

Given list:

```
Head
|
v
6 <-> 9 <-> 2
^         |
|         |
```

Call:

```
Length(Head);
```

Start:

```
p points to 6  
len = 0
```

Trace:

Step	p points to	Action	len
1	6	Count 6, move to 9	1
2	9	Count 9, move to 2	2
3	2	Count 2, move to 6	3
Stop	6	p == Head, stop	3

Final result:

```
Length = 3
```

17. Insertion Index Convention

Module 21 uses this insertion index convention:

Index	Meaning
0	Insert before Head. The new node becomes the new Head.
1	Insert after the first node.
2	Insert after the second node.
3	Insert after the third node.
n	Insert after the last node in a list of length n.

Example list:

```
6 <-> 9 <-> 2
```

This list has 3 nodes.

Valid insertion indexes:

0, 1, 2, 3

Meaning:

Index	Meaning if inserting 8
0	Insert before 6. New list starts with 8.
1	Insert after 6.
2	Insert after 9.
3	Insert after 2.

Invalid indexes:

-1, 4, 100

Validation rule:

```
if (index < 0 || index > Length(Head)) {  
    return;  
}
```

18. Insert Before Head

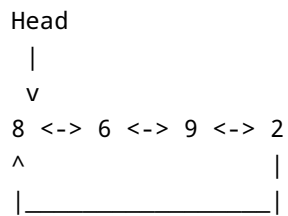
This is the most important operation in Module 21.

Suppose the list is:

```
Head  
|  
v  
6 <-> 9 <-> 2  
^         |  
|         |
```

We want to insert 8 before Head.

Expected result:



The new node **8** becomes the new **Head** .

19. Why Insert Before **Head** Is Fast

In a circular singly linked list, to insert before **Head** , we usually need to find the last node by traversal.

That means we may need to walk through the list:

```
Head -> node -> node -> last
```

But in a circular doubly linked list, the last node is immediately available using:

```
Head->prev
```

So we can write:

```
last = Head->prev;
```

This gives the last node directly.

Therefore, inserting before **Head** is:

```
O(1)
```

because we do not need to traverse the list.

20. Insert Before **Head** : Required Link Changes

Original list:

Head points to 6
last points to 2
new node t stores 8

Picture:

```
6 <-> 9 <-> 2
^         |
|         |
|         |
```

We need these links:

```
t->next = Head;
t->prev = last;
Head->prev = t;
last->next = t;
Head = t;
```

Meaning:

Code	Meaning
<code>t->next = Head;</code>	New node points forward to old first node.
<code>t->prev = last;</code>	New node points backward to old last node.
<code>Head->prev = t;</code>	Old first node points backward to new node.
<code>last->next = t;</code>	Old last node points forward to new node.
<code>Head = t;</code>	New node becomes the starting node.

Notice that the four real link fields being updated are:

```
t->next
t->prev
Head->prev
last->next
```

Then the external pointer `Head` is moved to `t`.

21. Insert Before Head : Empty List Case

Before insertion:

```
Head == NULL
```

There are no nodes.

Call:

```
InsertBeforeHead(10);
```

Create the new node:

```
10
```

Since it is the only node, it must point to itself in both directions.

Code pattern:

```
Head = t;  
Head->next = Head;  
Head->prev = Head;
```

Final structure:

```
Head  
|  
v  
10  
^|  
||  
|_|
```

Important:

A one-node circular doubly linked list does **not** use `NULL` for `prev` or `next`.

22. Insert Before Head: Code

```
void InsertBeforeHead(int x) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;

    // Case 1: empty list
    if (Head == NULL) {
        Head = t;
        Head->next = Head;
        Head->prev = Head;
        return;
    }

    // Case 2: non-empty list
    struct Node *last = Head->prev;

    t->next = Head;
    t->prev = last;

    Head->prev = t;
    last->next = t;

    Head = t;
}
```

Explanation

```
struct Node *t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
```

Create a new node and store the value inside it.

```
if (Head == NULL) {
    Head = t;
    Head->next = Head;
    Head->prev = Head;
    return;
}
```

If the list is empty, the new node becomes the only node. It points to itself in both directions.

```
struct Node *last = Head->prev;
```

Get the last node directly.

```
t->next = Head;  
t->prev = last;
```

Connect the new node to the old first node and old last node.

```
Head->prev = t;  
last->next = t;
```

Connect the old first and old last nodes to the new node.

```
Head = t;
```

Move `Head` to the new node.

23. Dry Run: Insert Before `Head`

Original list:

```
Head  
|  
v  
6 <-> 9 <-> 2  
^         |  
|_____|
```

Call:

```
InsertBeforeHead(8);
```

Step 1: Find the last node

```
last = Head->prev;
```

Since `Head` points to `6`, `Head->prev` points to `2`.

```
last -> 2
```

Step 2: New node points forward to old Head

```
t->next = Head;
```

Now:

```
8 -> 6
```

Step 3: New node points backward to last

```
t->prev = last;
```

Now:

```
2 <- 8 -> 6
```

Step 4: Old Head points backward to new node

```
Head->prev = t;
```

Now:

```
8 <-> 6
```

Step 5: Old last node points forward to new node

```
last->next = t;
```

Now:

```
2 <-> 8 <-> 6
```

Step 6: Move Head

```
Head = t;
```

Final list:

```
Head
|
v
8 <-> 6 <-> 9 <-> 2
^               |
|_____|
```

24. General Insertion After an Index

Insert before `Head` handles index `0`.

For any other index, we insert after a certain node.

Example list:

```
6 <-> 9 <-> 2
```

Call:

```
Insert(2, 7);
```

Using the index convention:

```
index 2 = insert after the second node
```

The second node is `9`.

Expected result:

```
6 <-> 9 <-> 7 <-> 2
```

25. General Four-Link Insertion Pattern

Suppose `p` points to the node after which we want to insert.

Before insertion:

```
p
|
v
9 <-> 2
```

We want to insert `7` after `9`.

The new node `t` must go between `p` and `p->next`.

Correct pattern:

```
t->prev = p;
t->next = p->next;
p->next->prev = t;
p->next = t;
```

Meaning:

Code	Meaning
<code>t->prev = p;</code>	New node points backward to <code>p</code> .
<code>t->next = p->next;</code>	New node points forward to the node after <code>p</code> .
<code>p->next->prev = t;</code>	The next node points backward to the new node.
<code>p->next = t;</code>	<code>p</code> points forward to the new node.

This is called a **four-link insertion**.

Why four links?

Because a circular doubly linked list must stay correct in both directions:

- forward direction through `next`,
- backward direction through `prev`.

26. Why the Order Matters in General Insertion

Correct order:

```
t->prev = p;  
t->next = p->next;  
p->next->prev = t;  
p->next = t;
```

The key line is:

```
t->next = p->next;
```

This saves the old next node before we overwrite `p->next`.

If we change `p->next` too early, we can lose the original node after `p`.

Wrong idea:

```
p->next = t;  
t->next = p->next;
```

After `p->next = t`, `p->next` no longer points to the old next node. It points to `t`.

Then `t->next = p->next` may make `t` point to itself.

Beginner rule:

```
Save the old connection before overwriting it.
```

27. Why End Insertion Also Works With the Same Pattern

In a linear doubly linked list, inserting after the last node needs a special case because:

```
p->next == NULL
```

So this would be unsafe:

```
p->next->prev = t;
```

But in a circular doubly linked list, the last node's `next` is not `NULL`.

For the last node:

```
p->next == Head
```

So the same four-link pattern works even when inserting after the last node.

Example:

```
6 <-> 9 <-> 2
^         |
|         |
|_____||
```

Insert after `2`.

Here:

```
p points to 2
p->next points to Head, which is 6
```

The new node is inserted between `2` and `6`, preserving the circle.

28. Complete Insert Function

```
void Insert(int index, int x) {
    int len = Length(Head);

    if (index < 0 || index > len) {
        return;
    }

    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;

    // Case 1: empty list
    if (Head == NULL) {
        if (index == 0) {
```

```

        Head = t;
        Head->next = Head;
        Head->prev = Head;
    } else {
        free(t);
    }
    return;
}

// Case 2: insert before Head
if (index == 0) {
    struct Node *last = Head->prev;

    t->next = Head;
    t->prev = last;

    Head->prev = t;
    last->next = t;

    Head = t;
    return;
}

// Case 3: insert after a position
struct Node *p = Head;

for (int i = 0; i < index - 1; i++) {
    p = p->next;
}

t->prev = p;
t->next = p->next;

p->next->prev = t;
p->next = t;
}

```

29. Explanation of the Complete Insert Function

Step 1: Get the length

```
int len = Length(Head);
```

We need the length to check whether the index is valid.

Step 2: Validate the index

```
if (index < 0 || index > len) {  
    return;  
}
```

If the index is invalid, do not change the list.

Step 3: Create the new node

```
struct Node *t = (struct Node *)malloc(sizeof(struct Node));  
t->data = x;
```

`t` points to the new node. The new node stores `x`.

Step 4: Empty list case

```
if (Head == NULL) {  
    if (index == 0) {  
        Head = t;  
        Head->next = Head;  
        Head->prev = Head;  
    } else {  
        free(t);  
    }  
    return;  
}
```

If the list is empty, only index `0` is valid.

The new node must point to itself:

```
Head->next = Head;  
Head->prev = Head;
```

Step 5: Insert before Head

```
if (index == 0) {
    struct Node *last = Head->prev;

    t->next = Head;
    t->prev = last;

    Head->prev = t;
    last->next = t;

    Head = t;
    return;
}
```

This inserts the new node before the current `Head`. The new node becomes the new `Head`.

Step 6: Insert after a position

```
struct Node *p = Head;

for (int i = 0; i < index - 1; i++) {
    p = p->next;
}
```

Move `p` to the node after which the new node should be inserted.

Then connect the new node:

```
t->prev = p;
t->next = p->next;
p->next->prev = t;
p->next = t;
```

30. Dry Run: Insert After an Index

Original list:

```
Head
|
```

```

      v
      8 <-> 6 <-> 9 <-> 2
      ^               |
      |_____|

```

Call:

```
Insert(2, 7);
```

Index **2** means insert after the second node.

The second node is **6**.

Step 1: Move **p**

Start:

```
p points to 8
```

Loop runs **index - 1 = 1** time.

After one move:

```
p points to 6
```

Step 2: Create node **7**

```
t -> 7
```

Step 3: Set backward link of new node

```
t->prev = p;
```

Now:

```
6 <- 7
```

Step 4: Set forward link of new node

```
t->next = p->next;
```

Since `p->next` is `9`, now:

```
7 -> 9
```

Step 5: Next node points back to new node

```
p->next->prev = t;
```

Now:

```
7 <- 9
```

Step 6: Previous node points forward to new node

```
p->next = t;
```

Now:

```
6 -> 7
```

Final list:

```
Head
|
v
8 <-> 6 <-> 7 <-> 9 <-> 2
^                               |
|_____|
```

31. Full Mini Program

```
#include <stdio.h>
#include <stdlib.h>
```

```

struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *Head = NULL;

int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}

void Display(struct Node *h) {
    if (h == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *p = h;

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != h);

    printf("\n");
}

void Insert(int index, int x) {
    int len = Length(Head);

    if (index < 0 || index > len) {
        return;
    }
}

```



```

struct Node *t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;

if (Head == NULL) {
    Head = t;
    Head->next = Head;
    Head->prev = Head;
    return;
}

if (index == 0) {
    struct Node *last = Head->prev;

    t->next = Head;
    t->prev = last;

    Head->prev = t;
    last->next = t;

    Head = t;
    return;
}

struct Node *p = Head;

for (int i = 0; i < index - 1; i++) {
    p = p->next;
}

t->prev = p;
t->next = p->next;

p->next->prev = t;
p->next = t;
}

int main() {
    Insert(0, 6);
    Insert(1, 9);
    Insert(2, 2);

    printf("Original circular doubly linked list: ");
    Display(Head);

    Insert(0, 8);
    printf("After inserting 8 before Head: ");
    Display(Head);
}

```

```

    Insert(2, 7);
    printf("After inserting 7 at index 2: ");
    Display(Head);

    return 0;
}

```

Expected output:

```

Original circular doubly linked list: 6 9 2
After inserting 8 before Head: 8 6 9 2
After inserting 7 at index 2: 8 6 7 9 2

```

32. Complexity Analysis

Operation	Time	Why
Display	$O(n)$	Must visit every node once.
Length	$O(n)$	Must count every node once.
Insert before <code>Head</code>	$O(1)$	Uses <code>Head->prev</code> to reach the last node instantly.
Insert after index	$O(n)$	Must move to the target position.
Extra space	$O(1)$	Uses only a few pointer variables.

Important point:

Circular doubly linked lists make head insertion faster than circular singly linked lists because the last node is directly available through:

```
Head->prev
```

33. Why Circular Doubly Head Insertion Is Better Than Circular Singly Head Insertion

In a circular singly linked list:

```
Head -> 6 -> 9 -> 2
^
|_____|
```

To insert before `Head`, we must update the last node's `next` pointer.

But a singly node has no `prev` pointer.

So to find the last node, we usually traverse:

```
while (p->next != Head) {
    p = p->next;
}
```

That takes $O(n)$ time.

In a circular doubly linked list:

```
last = Head->prev;
```

That gives the last node immediately.

So inserting before `Head` takes $O(1)$ time.

34. Common Mistakes

Mistake 1: Using `while (p != NULL)` for display

Wrong:

```
while (p != NULL)
```

Circular lists usually do not reach `NULL`.

Correct:

```
do {  
    ...  
} while (p != Head);
```

Mistake 2: Using `while (p != Head)` directly

Wrong:

```
struct Node *p = Head;  
while (p != Head) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

This does not print anything because `p == Head` at the start.

Correct:

```
do {  
    printf("%d ", p->data);  
    p = p->next;  
} while (p != Head);
```

Mistake 3: Forgetting `Head->prev`

In a circular doubly linked list:

```
Head->prev
```

must always point to the last node.

If this is broken, backward movement from `Head` will fail.

Mistake 4: Treating a one-node circular list like a linear doubly list

Wrong for circular doubly:

```
Head->prev = NULL;  
Head->next = NULL;
```

Correct:

```
Head->prev = Head;  
Head->next = Head;
```

Mistake 5: Updating only `next` links during insertion

In a singly linked list, this may be enough:

```
t->next = p->next;  
p->next = t;
```

But in a circular doubly linked list, this is incomplete.

You must update both directions:

```
t->prev = p;  
t->next = p->next;  
p->next->prev = t;  
p->next = t;
```

If you update only `next`, forward traversal may work, but backward traversal will be broken.

Mistake 6: Moving `Head` too early

Wrong idea:

```
Head = t;  
t->next = Head;
```

This may make the new node point to itself and lose the old list.

Correct idea:

```
t->next = Head;
t->prev = last;
Head->prev = t;
last->next = t;
Head = t;
```

Connect first. Then move `Head`.

Mistake 7: Forgetting the empty list case

This is unsafe when the list is empty:

```
struct Node *last = Head->prev;
```

If `Head == NULL`, then `Head->prev` is invalid.

So always check:

```
if (Head == NULL) {
    ...
}
```

before using `Head->prev`.

35. Edge Cases to Test

You should test these cases:

Case 1: Display an empty list

```
Head == NULL
```

Expected behavior:

```
Print nothing or print "List is empty".
```

Case 2: Insert into an empty list

Call:

```
Insert(0, 10);
```

Expected structure:

```
Head->data == 10;  
Head->next == Head;  
Head->prev == Head;
```

Case 3: Insert before Head in a one-node list

Before:

```
10
```

Call:

```
Insert(0, 5);
```

After:

```
5 <-> 10  
^      |  
|_____|
```

Important checks:

```
Head points to 5  
5->next points to 10  
5->prev points to 10  
10->next points to 5  
10->prev points to 5
```

Case 4: Insert before Head in a multi-node list

Before:

```
6 <-> 9 <-> 2
```

Call:

```
Insert(0, 8);
```

After:

```
8 <-> 6 <-> 9 <-> 2
```

Case 5: Insert in the middle

Before:

```
8 <-> 6 <-> 9 <-> 2
```

Call:

```
Insert(2, 7);
```

After:

```
8 <-> 6 <-> 7 <-> 9 <-> 2
```

Case 6: Insert after the last node

Before:

```
8 <-> 6 <-> 9 <-> 2
```

Call:


```
Insert(4, 10);
```

After:

```
8 <-> 6 <-> 9 <-> 2 <-> 10
^
|_____|
```

36. Quick Comparison With Other List Types

List type	Node links	End condition	Can move backward?	Head insertion cost
Linear singly	<code>next</code>	<code>NULL</code>	No	<code>O(1)</code>
Circular singly	<code>next</code>	Return to <code>Head</code>	No	Usually <code>O(n)</code> if last is not stored
Linear doubly	<code>prev</code> , <code>next</code>	<code>NULL</code> at both ends	Yes	<code>O(1)</code>
Circular doubly	<code>prev</code> , <code>next</code>	Return to <code>Head</code>	Yes	<code>O(1)</code> using <code>Head->prev</code>

Circular doubly linked lists are powerful because they support:

- forward movement,
- backward movement,
- circular traversal,
- fast access to the last node from `Head`.

The cost is extra memory because every node stores two pointers.

37. Module 21 Summary

A circular doubly linked list combines:

```
Circular behavior + Doubly linked behavior
```

The node structure is:

```
struct Node {  
    struct Node *prev;  
    int data;  
    struct Node *next;  
};
```

The main rules are:

```
last->next == Head;  
Head->prev == last;
```

A one-node circular doubly linked list has:

```
Head->next == Head;  
Head->prev == Head;
```

Display uses `do-while`:

```
do {  
    ...  
} while (p != Head);
```

Insert before `Head` is efficient because:

```
last = Head->prev;
```

So no traversal is needed.

General insertion needs four link updates:

```
t->prev = p;  
t->next = p->next;  
p->next->prev = t;  
p->next = t;
```

The core skill of Module 21 is:

Preserve the circle and preserve both directions.

38. Mastery Questions

You understand Module 21 if you can answer these questions:

1. What is a circular doubly linked list?
2. What are the three fields of a circular doubly linked-list node?
3. What does `prev` store?
4. What does `next` store?
5. What does `Head` represent?
6. Why does `last->next` point to `Head`?
7. Why does `Head->prev` point to the last node?
8. Why does display use `do-while`?
9. Why is `while (p != NULL)` wrong for circular display?
10. What are the conditions for an empty circular doubly linked list?
11. What are the conditions for a one-node circular doubly linked list?
12. Why is insert before `Head` $O(1)$?
13. What does `last = Head->prev` mean?
14. What four links are changed when inserting before `Head`?
15. What four links are changed in general middle insertion?
16. Why must we update both `prev` and `next` links?
17. What happens if we only update `next` links?
18. Why should we connect the new node before moving `Head`?
19. What is the time complexity of display?
20. What is the time complexity of insert before `Head`?
21. What is the time complexity of inserting after an index?
22. Why does circular doubly head insertion improve over circular singly head insertion?

39. Practice Tasks

Practice Task 1: Draw a one-node list

Draw the circular doubly linked list after:

```
Insert(0, 10);
```

Write the values of:

```
Head->data  
Head->next  
Head->prev
```

Expected idea:

```
Head->data == 10;  
Head->next == Head;  
Head->prev == Head;
```

Practice Task 2: Dry-run head insertion

Start with:

```
6 <-> 9 <-> 2
```

Dry-run:

```
Insert(0, 8);
```

Track these values:

Step	Head	last	t	Important link changed
Start	6	2	8	none
1	6	2	8	t->next = Head
2	6	2	8	t->prev = last
3	6	2	8	Head->prev = t
4	6	2	8	last->next = t
5	8	2	8	Head = t

Practice Task 3: Dry-run middle insertion

Start with:

```
8 <-> 6 <-> 9 <-> 2
```

Dry-run:

```
Insert(2, 7);
```

Final result should be:

```
8 <-> 6 <-> 7 <-> 9 <-> 2
```

Write which node `p` points to before linking.

Expected:

```
p points to 6
```

Practice Task 4: Check broken links

Suppose after insertion, forward display works:

```
8 6 7 9 2
```

But backward movement gives wrong results.

Question:

Which kind of links are probably broken?

Answer:

```
The prev links are probably broken.
```

40. Final Beginner Rule

Remember this simple rule:

```
A circular doubly linked list is a ring where every node knows both its next  
neighbor and its previous neighbor.
```

So every operation must keep two things correct:

1. The circle must remain closed.
2. The forward and backward links must agree.

If both are true, the circular doubly linked list is valid.